PETWELL

# PetWell — คำถามท้ายโปรเจกต์

สรุปการทำงานของระบบ 15 ข้อ

อธิบายสถาปัตยกรรมและการตัดสินใจเชิงเทคนิคของ PetWell โดยอ้างอิงจากโค้ดจริงในโปรเจกต์ทุกข้อ

Next.js 16 · NestJS 11 · PostgreSQL · Prisma 7 · TypeScript

กรกฎาคม 2026

Pet Health Passport & Appointment Management System

# Contents


**1 Flow การทำงานของโปรเจกต์** **4**

เส้นทางของ 1 request . . . . 4

จุดที่ควรสังเกต . . . . 4

**2 ปรับโครงสร้าง Database** **5**

แก้ schema.prisma . . . . 5

หลักที่ใช้ตัดสินใจ . . . . 5

สร้าง migration . . . . 5

**3 ปรับ code ฝั่ง NestJS** **5**

1. Request DTO — กำหนดว่ารับอะไรได้บ้าง . . . . 5

2. Response DTO — กำหนดว่าส่งอะไรออก . . . . 6

**4 ปรับ code ฝั่ง Next.js** **6**

การตัดสินใจด้าน UI . . . . 7

**5 การจัดเก็บและใช้งาน Token** **7**

เก็บที่ไหน . . . . 7

ส่งไป backend อย่างไร . . . . 7

ข้อดี . . . . 7

ความเสี่ยงที่ต้องยอมรับ . . . . 8

**6 ข้อมูลใดควรดึงจาก Server และข้อมูลใดจาก Client** **8**

ดึงบน Server Component . . . . 8

ต้องเป็น Client Component . . . . 8

**7 Image Component ของ Next.js ดีกว่า <img> อย่างไร** **8**

ผลต่อผู้ใช้ . . . . 9

สิ่งที่ต้องทำเพิ่มถ้าจะเปลี่ยน . . . . 9

**8 ทำไม Frontend ต้อง validate ทั้งที่ Backend ตรวจซ้ำ** **9**

หน้าที่ต่างกัน . . . . 9

เหตุผลสำคัญที่สุด . . . . 9

**9 providers, controllers, imports, exports ใน Module** **10**

จุดที่มักเข้าใจผิด . . . 10

**10 DTO ช่วยป้องกันปัญหาอะไร** **10**

ป้องกันอะไรได้บ้าง . . . 10

ถ้าไม่ใช้ DTO . . . 11

**11 Dependency Injection และ DI Container** **11**

เขียนแบบไหน . . . 11

ทำไมต้อง inject . . . 11

**12 ทำไม Controller ไม่ควรมี Business Logic** **11**

ผลกระทบถ้าเอา logic ไปไว้ใน controller . . . 12

**13 ลำดับการทำงานเมื่อ Request เข้ามาที่ NestJS** **12**

หน้าที่แต่ละส่วน ในโปรเจกต์นี้ . . . 12

**14 unknown ปลอดภัยกว่า any อย่างไร** **13**

ต่างกันตรงไหน . . . 13

ตัวอย่างจริงในโปรเจกต์ . . . 13

**15 Union Type และ Enum ต่างกันอย่างไร** **14**

Enum — ใช้เมื่อฐานข้อมูลบังคับ . . . 14

Union Type — ใช้เมื่อเป็นเรื่องของ TypeScript ล้วน . . . 14

เปรียบเทียบ . . . 14

สรุปแนวทางที่ใช้ . . . 14

# 1 Flow การทำงานของโปรเจกต์

ยกตัวอย่างจริง คือการทีเจ้าของสัตว์กดปุ่ม "Request appointment" ในหน้าจองนัด

## เส้นทางของ 1 request

```
BookAppointmentForm
  -> bookAppointmentAction
    -> AppointmentApi.create
      -> authFetch
        -> apiFetch
          => POST /appointments
              -> AuthGuard
              -> RolesGuard
              -> ValidationPipe
              -> AppointmentController.create
              -> AppointmentService.create
                  -> Prisma -> PostgreSQL
```

| ขั้นตอน | ทำอะไร |
|---|---|
| `BookAppointmentForm` | client component เก็บ state ของฟอร์ม |
| `bookAppointmentAction` | `'use server'` ตรวจด้วย zod ก่อนส่งออก |
| `AppointmentApi.create` | ห่อ path และ method ไว้ทีเดียว |
| `authFetch` | ดึง access token จาก session ใส่เป็น Bearer |
| `apiFetch` | fetch จริง และแปลง error เป็น `ApiError` |
| `AuthGuard` | มี token ไหม ทำงานแบบ global ทุก route ยกเว้นทีติด `@Public` |
| `RolesGuard` | ผู้เรียกเป็น OWNER ไหม |
| `ValidationPipe` | แปลง JSON เป็น `CreateAppointmentDto` และตัด field แปลกปลอม |
| `AppointmentController.create` | รับ `ownerId` จาก token แล้วเรียก service |
| `AppointmentService.create` | กฎธุรกิจทั้งหมดอยู่ตรงนี้ |

สิ่งที่ `AppointmentService.create` ตรวจตามลำดับ

- สัตว์เลี้ยงมีจริง เป็นของเจ้าของคนนี้ และยังไม่ถูก archive
- คลินิกมีจริง
- โควตา 2 นัดต่อชั่วโมงยังไม่เต็ม แล้วจึง insert ภายใน transaction เดียวกัน

ขากลับ service คืน object ธรรมดา NestJS แปลงเป็น JSON ส่งกลับ 201 ฝั่ง Next.js รับผลแล้ว `revalidatePath` เพื่อล้าง cache หน้ารายการนัด จากนั้น `router.push` ไปหน้า `/owner/appointments` ซึ่งเป็น Server Component ที่ดึงข้อมูลใหม่จาก API

## จุดที่ควรสังเกต

- **ฟอร์มไม่ได้ยิง API ตรง** ทุกอย่างผ่าน Server Action ทำให้ token ไม่เคยไปถึง browser

- **error เดินทางครบเส้น** ถ้าช่วงเวลาเต็ม backend ตอบ 409 -> `apiFetch` แปลงเป็น `ApiError` -> action คืน `{ success: false, message }` -> ฟอร์มแสดงข้อความนั้น ไม่ต้องเขียน handler เพิ่ม

# 2 ปรับโครงสร้าง Database

โจทย์ไม่ได้ระบุหัวข้อเฉพาะ จึงใช้งานจริงที่เพิ่งทำในโปรเจกต์เป็นตัวอย่าง คือ **การเพิ่มที่อยู่ของเจ้าของสัตว์** ซึ่งแตะครบทั้ง 3 ชั้น (ข้อ 2, 3, 4 ต่อเนื่องกัน)

## แก้ schema.prisma

```
model User {
  ...
  addressLine String? @map("address_line")
  subdistrict String?
  district    String?
  province    String?
  postalCode  String? @map("postal_code") @db.VarChar(5)
}
```

## หลักที่ใช้ตัดสินใจ

- **แยกเป็น 5 คอลัมน์ ไม่ใช่ก้อนเดียว** ที่อยู่ไทยมีโครงสร้างชัดเจน แยกเก็บแล้วยัง query ได้ เช่น นับเจ้าของแยกตามจังหวัด ถ้าเก็บเป็น text ก้อนเดียวต้องมา parse ใหม่ทีหลัง
- **ทุกคอลัมน์เป็น nullable** ผู้ใช้เดิมในระบบไม่ต้อง backfill และไม่บังคับให้กรอก
- `@map` **แปลงชื่อ** ฝั่ง TypeScript เป็น camelCase ฝั่ง SQL เป็น snake_case ตามธรรมเนียมของแต่ละภาษา
- `@db.VarChar(5)` รหัสไปรษณีย์ไทยมี 5 หลักเสมอ จำกัดที่ระดับฐานข้อมูลด้วย ไม่ใช่แค่ที่ validation

## สร้าง migration

```
pnpm prisma migrate dev --name add_owner_address
```

Prisma จะ generate SQL ให้ ไม่ควรเขียน SQL เอง เพราะจะทำให้ Prisma ตรวจพบ drift (schema กับ migration ไม่ตรงกัน)

> **migration คือประวัติ ไม่ใช่ไฟล์ทึง**
> ไฟล์ใน `prisma/migrations/` ต้อง commit เข้า git ทุกครั้ง เพราะเป็นสิ่งที่ทำให้เพื่อนร่วมทีมและ production ได้โครงสร้างเดียวกัน ส่วน `src/database/generated/` เป็นผลลัพธ์ที่ generate ใหม่ได้ จึงใส่ไว้ใน .gitignore

# 3 ปรับ code ฝั่ง NestJS

ต่อจากข้อ 2 เมื่อ schema เปลี่ยนแล้ว ฝั่ง API ต้องแก้ 2 จุด

## 1. Request DTO — กำหนดว่ารับอะไรได้บ้าง

```
export class UpdateProfileDto {
  @IsOptional()
  @ValidateIf((_, value) => value !== null)
  @IsString()
  @MaxLength(255)
  @TrimToNull()
  addressLine?: string | null;

  @IsOptional()
  @ValidateIf((_, value) => value !== null)
  @Matches(/^\d{5}$/, { message: 'postalCode must be 5 digits' })
  @TrimToNull()
  postalCode?: string | null;
}
```

จุดที่ต้องคิด คือ **ผู้ใช้ต้องลบข้อมูลได้** ฟอร์มที่ล้างช่องจะส่ง `""` มา ถ้าใช้ `@Trim()` เฉยๆ ค่าว่างจะถูกบันทึกลง database กลายเป็นคอลัมน์ที่ไม่ null แต่ไม่มีความหมาย จึงเขียน decorator `@TrimToNull()` ให้แปลง `""` เป็น `null` และใช้ `@ValidateIf` ข้ามการตรวจรูปแบบเมื่อค่าเป็น null มิฉะนั้นการลบรหัสไปรษณีย์จะไม่ผ่านกฎ 5 หลัก

### 2. Response DTO — กำหนดว่าส่งอะไรออก

```
export class UserResponseDto {
  addressLine: string | null;
  subdistrict: string | null;
  district: string | null;
  province: string | null;
  postalCode: string | null;
}
```

ส่วน service ไม่ต้องแก้เลย เพราะ `updateProfile` ส่ง dto เข้า Prisma ทั้งก้อนอยู่แล้ว

```
return this.prisma.user.update({
  where: { id: userId },
  data: dto,
  omit: { password: true },
});
```

ทีทำแบบนี้ได้อย่างปลอดภัย เพราะ `ValidationPipe` ตั้ง `whitelist: true` ไว้ ทำให้ field ที่ไม่มีใน DTO ถูกตัดทิ้งก่อนถึง service ผู้ใช้จึงแอบส่ง `role: 'CLINIC_STAFF'` มาแก้สิทธิ์ตัวเองไม่ได้

## 4 ปรับ code ฝั่ง Next.js

ต่อจากข้อ 3 ฝั่งหน้าเว็บแก้ 4 จุด

| ไฟล์ | แก้อะไร |
|---|---|
| `lib/api/api.type.ts` | เพิ่ม 5 field ใน `UserResponse` |
| `lib/schemas/user.schema.ts` | เพิ่มกฎ zod ให้ตรงกับฝั่ง backend |
| `lib/constants/provinces.ts` | รายชื่อ 77 จังหวัด สำหรับ dropdown |
| `components/features/user/ProfileForm.tsx` | เพิ่มส่วนกรอกที่อยู่ |

```
postalCode: z
  .string()
  .regex(/^$|^\d{5}$/, 'Postal code must be 5 digits.')
  .optional(),
```

สังเกตว่ากฎฝั่งหน้าเว็บยอมรับ `""` ด้วย เพราะค่าว่างแปลว่า "ลบข้อมูลนี้" ซึ่ง backend จะแปลงเป็น null ให้เอง

### การตัดสินใจด้าน UI

- **แสดงเฉพาะ Pet Owner** `ProfileForm` ถูกใช้ทั้งหน้า owner และ staff จึงเพิ่ม prop `showAddress` เพราะ Clinic Staff มีที่อยู่อยู่ที่ record ของคลินิกอยู่แล้ว
- **เมื่อไม่แสดง ต้องไม่ส่ง key นั้นไปด้วย** ถ้าส่ง `addressLine: ""` ไปตอน staff กดบันทึก ที่อยู่ที่เคยมีจะถูกลบ จึงตัด key ออกจาก payload ไปเลย
- **จังหวัดเป็น dropdown แต่ backend ไม่ล็อกรายชื่อ** dropdown ช่วยให้ข้อมูลสะอาด แต่ถ้าไป validate ตายตัวที่ backend วันไหนมีการเปลี่ยนชื่อเขตปกครอง ผู้ใช้จะบันทึกไม่ได้ทันที โดยไม่ได้เพิ่มความปลอดภัยจริง

# 5 การจัดเก็บและใช้งาน Token

### เก็บที่ไหน

ตอน login `AuthApi.login` ได้ `access_token` (JWT) กลับมา แล้วเก็บลงใน **session cookie ของ next-auth** ซึ่งเป็น **httpOnly** cookie

```
callbacks: {
  jwt({ token, user }) {
    if (user) token.access_token = user.access_token;
    return token;
  },
}
```

### ส่งไป backend อย่างไร

ทุกการเรียก API ผ่าน `authFetch` ซึ่งทำงาน **บน server เท่านั้น**

```
export async function authFetch<T>(path: string, options = {}) {
  const session = await auth();
  const token = session?.user?.access_token;
  if (!token) redirect('/login');
  return apiFetch<T>(path, { ...options, token });
}
```

จากนั้น `apiFetch` ใส่ header `Authorization: Bearer <token>`

### ข้อดี

- **JavaScript ในหน้าเว็บอ่าน token ไม่ได้เลย** เพราะเป็น httpOnly cookie ต่อให้มีช่องโหว่ XSS ก็ขโมย token ไปใช้ต่อไม่ได้
- **token ไม่เคยอยู่ใน bundle ฝั่ง client** เพราะ fetch เกิดบน server ทั้งหมด
- **API ไม่ต้องเก็บ session** JWT พกข้อมูลตัวเองมาครบ ขยาย server ได้ง่าย

### ความเสี่ยงที่ต้องยอมรับ

- **ยกเลิก token กลางคันไม่ได้** JWT ใช้ได้จนกว่าจะหมดอายุ ถ้า token รั่วก็ต้องรอหมดอายุ (`ACCESS_TOKEN_EXPIRES_IN`) โปรเจกต์นี้ยังไม่มี refresh token rotation
- **ข้อมูลใน token ค้างได้** เจอมาแล้วจริงในโปรเจกต์ คือชื่อบน header ไม่ตรงกับหน้า Profile เพราะ header อ่านจาก token ที่ตั้งตอน login ส่วนหน้า Profile ดึงสดจาก `/auth/me` แก้โดยเรียก `unstable_update` หลังบันทึกโปรไฟล์สำเร็จ เพื่อเขียนค่าใหม่ลง token
- **cookie ต้องกัน CSRF** next-auth จัดการให้ และ backend ตั้ง CORS จำกัดเฉพาะ origin ของหน้าเว็บ

# 6 ข้อมูลใดควรดึงจาก Server และข้อมูลใดจาก Client

หลักง่ายๆ คือ **ถ้าไม่ต้องรอการกดของผู้ใช้ ให้ดึงบน server**

### ดึงบน Server Component

| หน้า | ทำไม |
|---|---|
| `/owner/pets` รายการสัตว์เลี้ยง | เป็นข้อมูลชุดแรกที่ต้องเห็นทันที ดึงบน server แล้วส่ง HTML ที่มีข้อมูลครบไปเลย ไม่มีอาการกระพริบ |
| `/owner/appointments/[id]` รายละเอียดนัด | ต้องใช้ token ซึ่งอยู่ฝั่ง server อยู่แล้ว |
| `/staff/pets` ค้นหาผู้ป่วย | อ่านคำค้นจาก `searchParams` ทำให้ URL แชร์ได้และ refresh แล้วไม่หาย |

ข้อดีที่สำคัญ คือ **ไม่ต้องส่ง token ออกไปที่ browser** และไม่ต้องเขียน state สำหรับ loading

### ต้องเป็น Client Component

| component | ทำไม |
|---|---|
| `BookAppointmentForm` | ต้องเก็บค่าที่ผู้ใช้พิมพ์ ใช้ `useState` |
| `SlotPicker` | ต้องดึงช่วงเวลาว่างใหม่ทุกครั้งที่เปลี่ยนคลินิกหรือวันที่ ซึ่งเกิดหลังหน้าโหลดแล้ว |
| `ProfileMenu` | ต้องเปิด/ปิดเมนู และดักคลิกนอกกรอบ |
| `StatusActions` | ต้องมี `onClick` และแสดงสถานะกำลังบันทึก |

> **Client Component ไม่ได้แปลว่าต้องยิง API เอง**
> `SlotPicker` เป็น client component แต่ไม่ได้ fetch เอง มันเรียก `getSlotAvailabilityAction` ซึ่งเป็น Server Action ทำให้ token ยังอยู่ฝั่ง server เหมือนเดิม รูปแบบนี้ใช้ทั่วทั้งโปรเจกต์

# 7 Image Component ของ Next.js ดีกว่า `<img>` อย่างไร

ปัจจุบันโปรเจกต์ยังใช้ `<img>` อยู่ 5 จุด (รูปสัตว์เลี้ยง รูปโปรไฟล์) และมี `eslint-disable` กำกับไว้ ถ้าเปลี่ยนมาใช้ `next/image` จะได้ประโยชน์ดังนี้

| ด้าน | `<img>` | `next/image` |
|---|---|---|
| ขนาดไฟล์ | ส่งไฟล์ต้นฉบับเต็ม | ย่อขนาดและแปลงเป็น WebP/AVIF อัตโนมัติ |
| ความเร็ว | โหลดทุกรูปพร้อมกัน | lazy load เฉพาะรูปที่กำลังจะเห็น |
| การกระตุก | หน้าเลื่อนเมื่อรูปโหลดเสร็จ | จองพื้นที่ไว้ก่อนจาก width/height |
| จอความละเอียดสูง | ต้องทำ srcset เอง | สร้าง srcset ให้ตามขนาดจอ |
| cache | ตาม header ต้นทาง | มี cache ของ Next เพิ่มอีกชั้น |

### ผลต่อผู้ใช้

รูปสัตว์เลี้ยงที่เจ้าของถ่ายจากมือถือมักมีขนาด 3–5 MB แต่แสดงจริงแค่ 400px `next/image` จะส่งเฉพาะขนาดที่ใช้จริง ซึ่งอาจเล็กลงกว่า 10 เท่า สำคัญมากกับผู้ใช้บนมือถือที่ใช้เน็ตมือถือ

ส่วนการจองพื้นที่ล่วงหน้าช่วยแก้อาการ **หน้าเว็บกระโดด** ตอนรูปโหลดเสร็จ ซึ่งเป็นสาเหตุที่ทำให้ผู้ใช้กดผิดปุ่ม

### สิ่งที่ต้องทำเพิ่มถ้าจะเปลี่ยน

ต้องประกาศ domain ของ Cloudinary ใน `next.config.ts` ก่อน เพราะ Next จะไม่ยอม optimize รูปจากโดเมนที่ไม่ได้อนุญาต

```
images: { remotePatterns: [{ hostname: 'res.cloudinary.com' }] }
```

# 8 ทำไม Frontend ต้อง validate ทั้งที่ Backend ตรวจซ้ำ

สองฝั่งทำคนละหน้าที่ ไม่ใช่งานซ้ำซ้อน

### หน้าที่ต่างกัน

| ฝั่ง | หน้าที่ | ถ้าขาดจะเป็นอย่างไร |
|---|---|---|
| Frontend | ให้ผู้ใช้รู้ตัวเร็ว | ต้องรอ network ไป-กลับ กว่าจะรู้ว่าพิมพ์ผิด |
| Backend | ปกป้องข้อมูลจริง | ข้อมูลเสียหายได้ทันที |

### เหตุผลสำคัญที่สุด

**Frontend validation ข้ามได้** ใครก็ตามที่เปิด terminal แล้วยิง

```
curl -X POST http://localhost:8000/appointments -d '{"appointmentDate":"1999-01-01"}'
```

จะไม่ผ่านหน้าเว็บเลยแม้แต่นิดเดียว ฉะนั้นกฎที่อยู่แต่ในหน้าเว็บ **ไม่ใช่กฎ** แต่เป็นแค่คำแนะนำ ในโปรเจกต์นี้ `@MinDate` ที่ backend คือกฎจริง ส่วนปฏิทินที่ปิดวันย้อนหลังคือความสะดวก

ตัวอย่างจากโปรเจกต์ ที่เห็นบทบาทของทั้งสองฝั่งชัดเจนที่สุดคือโควตาการจอง

- **หน้าเว็บ** เทาช่วงเวลาที่เต็มแล้ว ผู้ใช้จึงไม่เลือกไปชนกำแพง
- **backend** ยังนับและปฏิเสธด้วย 409 อยู่ดี เพราะระหว่างที่ผู้ใช้กำลังกรอกฟอร์ม อาจมีคนอื่นจองที่นั้งสุดท้ายไปแล้ว

> **จุดที่ยังต้องปรับปรุง**
> `POST /appointments` ยังไม่ได้ตรวจเวลาทำการ หน้าเว็บให้เลือกได้แค่ 09:00–17:00 แต่ถ้ายิง API ตรงยังจองตี 3 ได้ เป็นตัวอย่างที่ดีของกฎที่อยู่แต่ฝั่งหน้าเว็บ

# 9 providers, controllers, imports, exports ใน Module

```
@Module({
  controllers: [AppointmentController],
  providers: [AppointmentService],
  exports: [AppointmentService],
})
export class AppointmentModule {}
```

| ส่วน | หน้าที่ | ตัวอย่างในโปรเจกต์ |
|---|---|---|
| `controllers` | class ที่รับ HTTP request | `AppointmentController` map `POST /appointments` ไปที่ method |
| `providers` | class ที่ inject ได้ ใช้ได้ในโมดูลนี้ | `AppointmentService` เก็บกฎธุรกิจทั้งหมด |
| `imports` | ดึงของจากโมดูลอื่นมาใช้ | `PetModule` import `UploadModule` เพื่อใช้ `CloudinaryService` |
| `exports` | เปิดของในโมดูลนี้ให้โมดูลอื่นใช้ | `AuthModule` ต้องใช้ `UserService` ได้เพราะ `UserModule` export ไว้ |

## จุดที่มักเข้าใจผิด

ถ้าไม่ `exports` โมดูลอื่นจะ inject ไม่ได้ แม้จะ `imports` มาแล้วก็ตาม ทั้งสองอย่างต้องคู่กัน

ส่วน `DatabaseModule` ใช้ `@Global()` จึงไม่ต้อง import ซ้ำทุกโมดูล เพราะ `PrismaService` ควรมี connection pool เดียวทั้งแอป

```
@Global()
@Module({ providers: [PrismaService], exports: [PrismaService] })
export class DatabaseModule {}
```

# 10 DTO ช่วยป้องกันปัญหาอะไร

DTO คือ class ที่บอกว่า "ข้อมูลที่วิ่งข้ามขอบเขต network หน้าตาเป็นอย่างไร" ในโปรเจกต์แบ่งเป็น 2 แบบ **Request DTO** มี decorator ตรวจค่า ส่วน **Response DTO** เป็น class เปล่าที่บอกว่าจะส่งอะไรออกไป

## ป้องกันอะไรได้บ้าง

| ปัญหา | DTO ช่วยอย่างไร |
|---|---|
| Mass assignment | `whitelist: true` ตัด field ที่ไม่ได้ประกาศทิ้ง ส่ง `role` มาเองไม่ได้ |

| ปัญหา | DTO ช่วยอย่างไร |
| --- | --- |
| ข้อมูลผิดรูปแบบ | `@IsUUID()` `@Matches(/^\d{5}$/)` ปฏิเสธตั้งแต่ก่อนถึง service |
| ชนิดข้อมูลไม่ตรง | `@Type(() => Date)` แปลง string เป็น Date จริง service จึงไม่ต้องเดา |
| ข้อมูลลับรั่ว | `UserResponseDto` ไม่มี field `password` เลย |

### ถ้าไม่ใช้ DTO

ถ้า controller รับ `@Body() body: any` แล้วส่งเข้า Prisma ตรงๆ ผลคือ

1. ใครส่ง `{ "role": "CLINIC_STAFF" }` มาก็เลื่อนขั้นตัวเองได้ 2. `appointmentDate` เป็น string ที Prisma ปฏิเสธ กลายเป็น error 500 แทนทีจะเป็น 400 3. เพิมคอลัมน์ใหม่ใน database แล้วมันไหลออก API ทันทีโดยไม่มีใครตั้งใจ 4. อ่านโค้ดแล้วไม่รู้ว่า endpoint นีรับอะไรได้บ้าง ต้องไล่โค้ดทังเส้น

## 11 Dependency Injection และ DI Container

### เขียนแบบไหน

```
@Injectable()
export class AppointmentService {
  constructor(private readonly prisma: PrismaService) {}
}
```

ไม่ใช่แบบนี

```
export class AppointmentService {
  private prisma = new PrismaService();
}
```

### ทำไมต้อง inject

- **ไม่เปิด connection ซ้ำ** ถ้าแต่ละ service `new PrismaService()` เอง จะได้ connection pool คนละอันเต็มไปหมด DI container สร้างครังเดียวแล้วแจกให้ทุกที
- **เขียน test ได้จริง** ตอนทดสอบ service ในโปรเจกต์นี ส่ง object ปลอมทีมีแค่ method ทีต้องใช้เข้าไปแทน Prisma จริงได้เลย ทำให้ทดสอบกฎโควตาและกฎการเปลียนสถานะได้โดยไม่ต้องมีฐานข้อมูล
- **เปลียนของข้างในได้โดยไม่แก้ผู้ใช้** ถ้าวันหนึงเปลียนจาก bcrypt เป็น argon2 แก้แค่ `BcryptService` ทีเดียว ส่วน `AuthService` ไม่ต้องแตะ
- **จัดลำดับการสร้างให้เอง** `AppointmentService` ต้องการ `PrismaService` ซึงต้องการ `ConfigService` DI container ไล่ลำดับให้ครบเอง

## 12 ทำไม Controller ไม่ควรมี Business Logic

Controller ควรทำแค่ 3 อย่าง คือ รับ request ดึงค่าทีต้องใช้ แล้วเรียก service

```
@Post()
create(
  @CurrentUser('sub') ownerId: string,
  @Body() dto: CreateAppointmentDto,
): Promise<AppointmentResponseDto> {
  return this.appointmentService.create(ownerId, dto);
}
```

### ผลกระทบถ้าเอา logic ไปไว้ใน controller

**ใช้ซ้ำไม่ได้** กฎ "ยกเลิกนัดเมื่อ archive สัตว์เลี้ยง" อยู่ใน `PetService.archive` ถ้าเขียนไว้ใน controller เมื่อมี job อัตโนมัติที่ต้อง archive ด้วยจะต้องคัดลอกโค้ดไปอีกที

**test ยากขึ้นมาก** ทดสอบ service ทำได้โดยสร้าง class ขึ้นมาแล้วเรียก method ตรงๆ แต่ทดสอบ controller ที่มี logic ต้องจำลอง HTTP ทั้งชุด ในโปรเจกต์นี้ทดสอบกฎการเปลี่ยนสถานะ 18 เคสได้เพราะ logic อยู่ใน service ล้วน

**หาโค้ดไม่เจอ** ถ้าอยากรู้ว่า "นัดหมายเปลี่ยนสถานะอย่างไรได้บ้าง" ควรเปิดไฟล์เดียวจบ ไม่ใช่ไล่อ่านทั้ง controller และ service

**เปลี่ยนช่องทางไม่ได้** ถ้าจะเพิ่ม CLI หรือ message queue ที่ทำงานเดียวกัน logic ที่ผูกกับ HTTP ใช้ต่อไม่ได้เลย

# 13 ลำดับการทำงานเมื่อ Request เข้ามาที่ NestJS

```
Request
  -> Middleware
  -> Guard
  -> Interceptor (before)
  -> Pipe
  -> Controller
  -> Service
  -> Interceptor (after)
  -> Response
```

| ลำดับ | ทำอะไร |
|---|---|
| Middleware | CORS |
| Guard | ตรวจสิทธิ์ ผ่านหรือโยน exception |
| Interceptor | ทำงานก่อนเข้า handler |
| Pipe | แปลงและตรวจค่า argument |
| Controller | เรียก service |
| Service | กฎธุรกิจและติดต่อ Prisma |
| Interceptor | ทำงานหลังออกจาก handler |

### หน้าที่แต่ละส่วน ในโปรเจกต์นี้

| ส่วน | ตัวอย่าง | ทำอะไร |
|---|---|---|
| Guard | `AuthGuard` | ลงทะเบียนแบบ global จึง**ปิดทุก route โดยปริยาย** route สาธารณะต้องติด `@Public()` เอง |
| Guard | `RolesGuard` | ทำงานต่อจาก `AuthGuard` จึงอ่าน `request.user` ได้เลย เช็คว่า role ตรงกับ `@Roles()` ไหม |
| Pipe | `ValidationPipe` | แปลง JSON เป็น DTO ตัด field แปลกปลอม ตรวจกฎทั้งหมด |
| Pipe | `ParseUUIDPipe` | id ที่ผิดรูปแบบตอบ 400 ตั้งแต่ก่อนแตะฐานข้อมูล |
| Controller | `AppointmentController` | ดึง `ownerId` จาก token แล้วส่งต่อ |
| Service | `AppointmentService` | ตรวจความเป็นเจ้าของ นับโควตา เขียน audit log |

> **ทำไมต้องปิดทุก route โดยปริยาย**
> ถ้าเลือกทางกลับกัน คือติด decorator เฉพาะ route ที่ต้องป้องกัน วันที่ลืมติดจะได้ endpoint ที่เปิดโล่งโดยไม่มีสัญญาณเตือนใดๆ แต่แบบที่ใช้อยู่ ถ้าลืมติด `@Public()` route สาธารณะจะตอบ 401 ซึ่งเห็นทันที ความผิดพลาดจึงตกไปในทางที่ปลอดภัย

# 14 unknown ปลอดภัยกว่า any อย่างไร

## ต่างกันตรงไหน

| | `any` | `unknown` |
|---|---|---|
| ใช้ค่าได้เลยไหม | ได้ทันที | ไม่ได้ ต้องตรวจชนิดก่อน |
| TypeScript ตรวจให้ไหม | ปิดการตรวจทั้งหมด | ยังตรวจให้ตามปกติ |
| error ไปโผล่ตอนไหน | ตอนรันจริง | ตอน compile |

## ตัวอย่างจริงในโปรเจกต์

```
export function TrimToNull() {
  return Transform(({ value }: { value: unknown }) => {
    if (typeof value !== 'string') return value;
    const trimmed = value.trim();
    return trimmed === '' ? null : trimmed;
  });
}
```

ค่าที่เข้ามาคือสิ่งที่ผู้ใช้ส่งมา จะเป็น string, number, array หรือ null ก็ได้ การประกาศเป็น `unknown` ทำให้ TypeScript **บังคับ** ให้เช็ค `typeof value !== 'string'` ก่อน ถ้าใช้ `any` แล้วเผลอเรียก `value.trim()` โดยตรง โค้ดจะ compile ผ่านเงียบๆ แล้วพังตอนมีคนส่งตัวเลขมา

พูดง่ายๆ คือ `any` แปลว่า "ไม่ต้องตรวจ" ส่วน `unknown` แปลว่า "ยังไม่รู้ว่าอะไร ไปตรวจก่อน"
ตัวหลังตรงกับความจริงของข้อมูลจากภายนอกมากกว่า

# 15 Union Type และ Enum ต่างกันอย่างไร

โปรเจกต์นี้ใช้ทั้งสองแบบ โดยแบ่งตามที่มาของข้อมูล

## Enum — ใช้เมื่อฐานข้อมูลบังคับ

```
enum AppointmentStatus {
  PENDING
  CONFIRMED
  REJECTED
  RESCHEDULED
  COMPLETED
  CANCELLED
}
```

Prisma generate เป็น TypeScript enum ให้ ฝั่ง NestJS จึงเขียน `AppointmentStatus.CONFIRMED` ได้ ข้อดีคือ **ฐานข้อมูลปฏิเสธค่าที่ไม่มีในรายการ** ต่อให้ code มีบั๊กก็เขียนค่าเพี้ยนลงไปไม่ได้

## Union Type — ใช้เมื่อเป็นเรื่องของ TypeScript ล้วน

```
export type AppointmentStatus =
  | 'PENDING' | 'CONFIRMED' | 'REJECTED'
  | 'RESCHEDULED' | 'COMPLETED' | 'CANCELLED';
```

ฝั่ง Next.js ใช้แบบนี้ เพราะรับค่ามาเป็น JSON string อยู่แล้ว ไม่ต้อง import อะไรจาก Prisma และไม่มีโค้ดเหลืออยู่ใน bundle เลยหลัง compile

## เปรียบเทียบ

| | Enum | Union Type |
|---|---|---|
| หลัง compile | เหลือ object จริงใน bundle | หายไปหมด |
| ใช้กับ decorator | ได้ เช่น `@IsEnum(AppointmentStatus)` | ไม่ได้ |
| วนลูปค่าทั้งหมด | ได้ | ต้องประกาศ array แยก |
| เทียบกับ string ตรงๆ | ต้องอ้างผ่าน enum | เทียบได้เลย |

## สรุปแนวทางที่ใช้

เลือก **enum** เมื่อค่านั้นต้องมีอยู่จริงตอนรัน เช่น ต้องส่งให้ `@IsEnum()` หรือต้อง map กับคอลัมน์ในฐานข้อมูล และเลือก **union type** เมื่อต้องการแค่ให้ TypeScript ตรวจให้ตอน compile เช่น prop ของ component หรือ type ของ response ที่รับมาจาก API